Lab Eleven – Point Estimation and Resampling

Tips

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with "**Solution**:".
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.
- Make sure to use "enter" to make your code readable, so it doesn't run off the page. I switched the Quarto document to automatically render to .pdf, so you can see what you're submitting by default. You can switch back for the highlighting by moving the html block above the pdf block in the YAML header.
- Don't forget that you can copy-and-paste code when you're doing something similar as we did in class!
- Make sure to plot all computed probabilities. This is good `ggplot` practice AND will help make sure you don't make a mistake when computing the probability.

1 Introduction

When performing point estimation, there isn't just one *right* answer. For example, the Method of Moments (MOM) and Maximum Likelihood Estimates (MLEs) aren't always the same.

One question that comes up, is why we calculate the sample variance as

$$S_{n-1}^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2,$$

when it is supposed to be the "average squared distance from the mean, i.e.,

$$S_n^2 = \frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2.$$

2 Comparing Estimators

2.1 Initial Simulation

In this lab, we will explore why we use S_{n-1}^2 and not S_n^2. To do this, we will generate data from four known distributions:

- Poisson$(\lambda = 2)$
- Binomial$\left(n = 50, p = \frac{5-\sqrt{21}}{10}\right)$
- Normal$(\mu = 0, \sigma = \sqrt{2})$
- Exponential$\left(\lambda = \frac{1}{\sqrt{2}}\right)$.

Remark: Note that I have selected the parameters so that the true variance for all four models is 2.

For each distribution, write a loop that completes the following 1000 times. Put `set.seed(7272)` at the top of your code chunk.

- Generate a sample of size n from the distribution.
- Calculate and save S_{n-1}^2 and S_n^2.

Start with $n = 20$. Is there evidence that S_{n-1}^2 is a better estimator than S_n^2?

Solution

```r
set.seed(7272)

#Poisson Distribution
  psv = c()
  pad = c()
for(i in 1:1000){
  x = rpois(20, 2)
  psv[i] = var(x)
  pad[i] = ((20-1)/20)*var(x)
}
mean(psv)
```

```
[1] 1.983863
```

```r
mean(pad)
```

```
[1] 1.88467
```

```r
#Binomial Distribution
  bsv = c()
  bad = c()
for(i in 1:1000){
  x = rbinom(20, 50, (5-sqrt(21))/10)
  bsv[i] = var(x)
  bad[i] = ((20-1)/20)*var(x)
}
mean(psv)
```

```
[1] 1.983863
```

```r
mean(pad)
```

```
[1] 1.88467
```

```r
#Normal Distribution
  nsv = c()
  nad = c()
for(i in 1:1000){
  x = rnorm(20, 0, sqrt(2))
  nsv[i] = var(x)
  nad[i] = ((20-1)/20)*var(x)
}
mean(nsv)
```

```
[1] 2.00228
```

```r
mean(nad)
```

```
[1] 1.902166
```

```r
#Exponential Distribution
  esv = c()
  ead = c()
for(i in 1:1000){
  x = rexp(20, 1/(sqrt(2)))
  esv[i] = var(x)
  ead[i] = ((20-1)/20)*var(x)
}
mean(esv)
```

```
[1] 1.984202
```

```r
mean(ead)
```

```
[1] 1.884992
```

There is evidence that S_{n-1}^2 is a better estimator than S_n^2 as S_{n-1}^2 is closer to the true variance of two by about 10%.

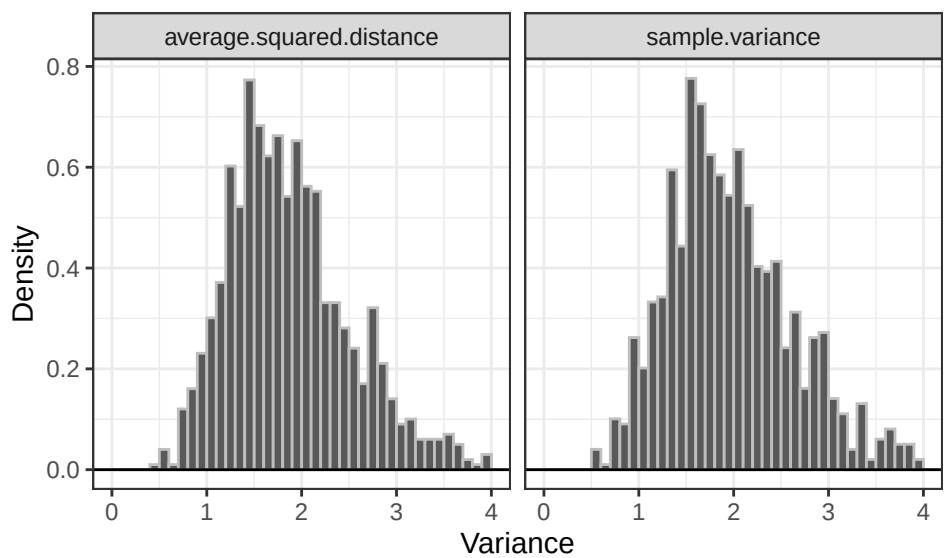
Create a plot or combination of plots (using patchwork) to show the results across all four distributions. Make sure to create a corresponding table that numerically summarizes the results of the simulation. You should include

$$\text{Bias} = \text{Average of Estimates} - \text{True Value}$$
$$\text{Precision} = \frac{1}{\text{Variance of Estimates}}$$
$$\text{Mean Square Error} = \text{Bias}^2 + \text{Variance of Estimates.}$$

Solution

```r
ggdat.p <- tibble(sample.variance = psv,
                  average.squared.distance = pad) |>
 pivot_longer(everything(),
 names_to = "Estimator",
 values_to = "Estimate")

ggplot(ggdat.p) +
 geom_histogram(aes(x = Estimate,
                    y = after_stat(density)),
                    breaks=seq(0,4,0.1), color="grey")+
 geom_hline(yintercept=0)+
 theme_bw() +
 labs(x= "Variance",
      y="Density") +
 facet_wrap(~Estimator)
```

Figure 1: Poisson Variance Estimation.




```r
P.Estimate.Summary = ggdat.p|>
  group_by(Estimator) |>
  summarize(
    Bias = mean(Estimate) - 2,
    Mean = mean(Estimate),
    Precision = 1/(var(Estimate)),
    MSE = Bias^2 + var(Estimate),
  )
```

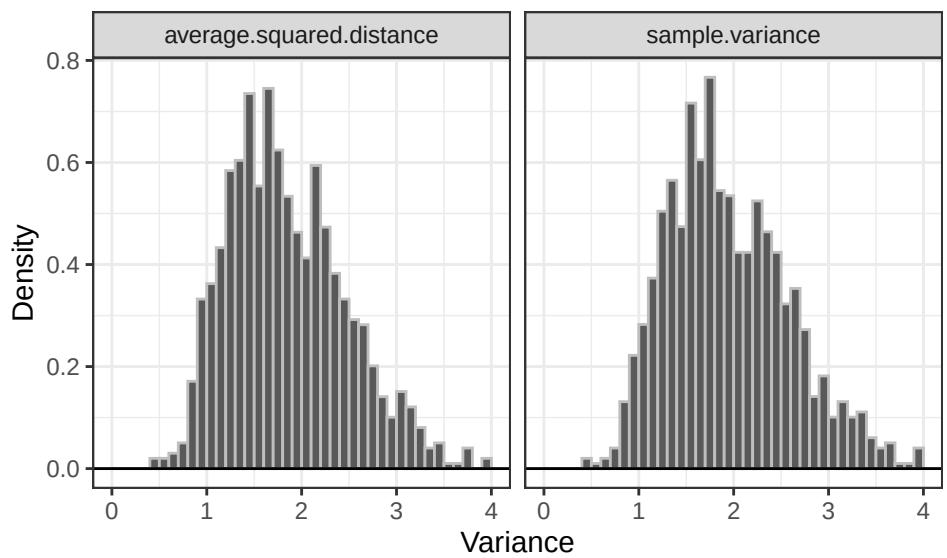
```r
P.Estimate.Summary |>
  knitr::kable(digits = 2)
```

Table 1: Poisson-Variance-Estimates.

Estimator	Bias	Mean	Precision	MSE
average.squared.distance	-0.12	1.88	2.33	0.44
sample.variance	-0.02	1.98	2.10	0.48

```r
ggdat.b <- tibble(sample.variance = bsv,
                  average.squared.distance = bad) |>
 pivot_longer(everything(),
 names_to = "Estimator",
 values_to = "Estimate")

ggplot(ggdat.b) +
 geom_histogram(aes(x = Estimate,
                    y = after_stat(density)),
                    breaks=seq(0,4,0.1), color="grey")+
 geom_hline(yintercept=0)+
 theme_bw() +
 labs(x= "Variance",
      y="Density") +
 facet_wrap(~Estimator)
```

Figure 2: Binomial Variance Estimation.




```r
B.Estimate.Summary = ggdat.b|>
  group_by(Estimator) |>
  summarize(
    Bias = mean(Estimate) - 2,
    Mean = mean(Estimate),
    Precision = 1/(var(Estimate)),
    MSE = Bias^2 + var(Estimate),
  )
```

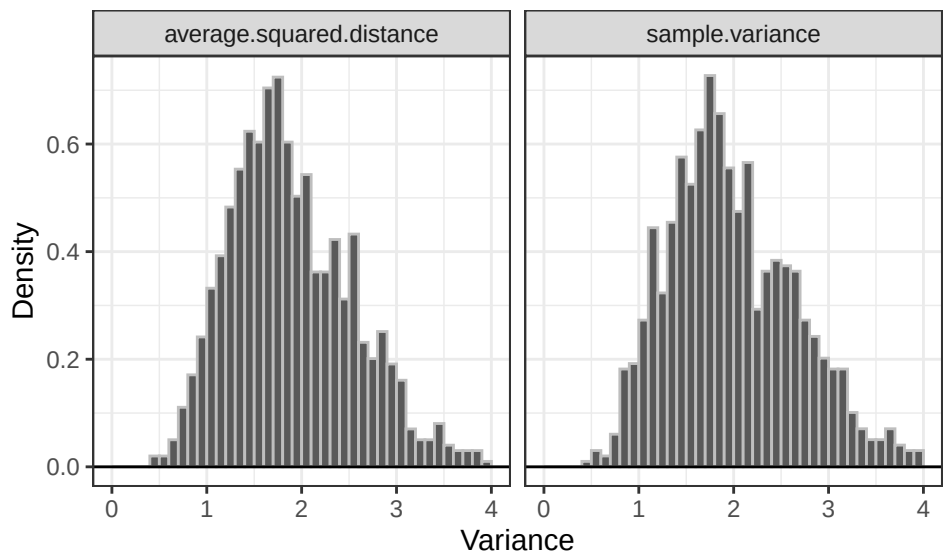
```r
B.Estimate.Summary |>
  knitr::kable(digits = 2)
```

Table 2: Binomial-Variance-Estimates.

Estimator	Bias	Mean	Precision	MSE
average.squared.distance	-0.13	1.87	2.38	0.44
sample.variance	-0.04	1.96	2.15	0.47

```
ggdat.n <- tibble(sample.variance = nsv,
                  average.squared.distance = nad) |>
 pivot_longer(everything(),
 names_to = "Estimator",
 values_to = "Estimate")

ggplot(ggdat.n) +
 geom_histogram(aes(x = Estimate,
                    y = after_stat(density)),
                    breaks=seq(0,4,0.1), color="grey")+
 geom_hline(yintercept=0)+
 theme_bw() +
 labs(x= "Variance",
      y="Density") +
 facet_wrap(~Estimator)
```

Figure 3: Normal Variance Estimation.




```
N.Estimate.Summary = ggdat.n|>
  group_by(Estimator) |>
  summarize(
    Bias = mean(Estimate) - 2,
    Mean = mean(Estimate),
    Precision = 1/(var(Estimate)),
    MSE = Bias^2 + var(Estimate),
  )
```

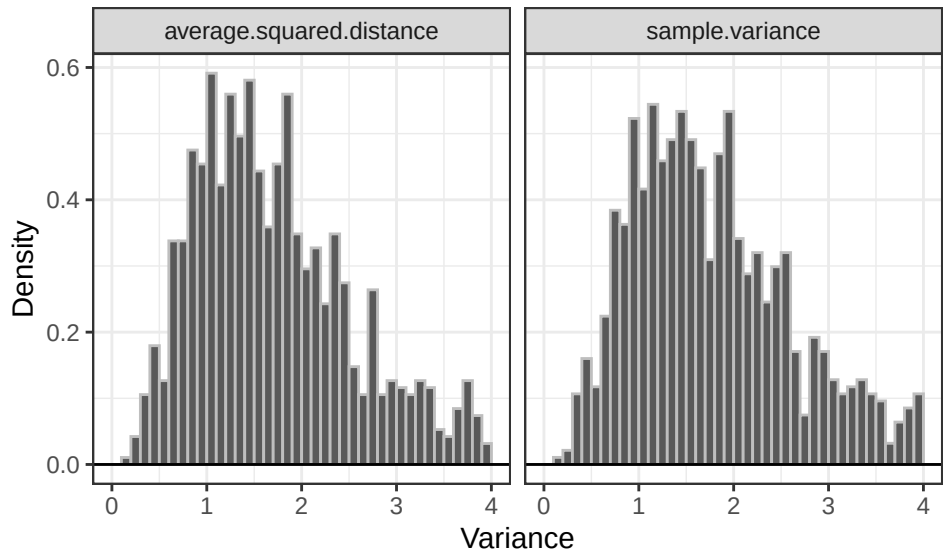
```
N.Estimate.Summary |>
  knitr::kable(digits = 2)
```

Table 3: Normal-Variance-Estimates.

Estimator	Bias	Mean	Precision	MSE
average.squared.distance	-0.1	1.9	2.25	0.45
sample.variance	0.0	2.0	2.03	0.49

```r
ggdat.e <- tibble(sample.variance = esv,
                  average.squared.distance = ead) |>
 pivot_longer(everything(),
 names_to = "Estimator",
 values_to = "Estimate")

ggplot(ggdat.e) +
 geom_histogram(aes(x = Estimate,
                    y = after_stat(density)),
                    breaks=seq(0,4,0.1), color="grey")+
 geom_hline(yintercept=0)+
 theme_bw() +
 labs(x= "Variance",
      y="Density") +
 facet_wrap(~Estimator)
```

Figure 4: Exponential Variance Estimation.




```r
E.Estimate.Summary = ggdat.e|>
  group_by(Estimator) |>
  summarize(
    Bias = mean(Estimate) - 2,
    Mean = mean(Estimate),
    Precision = 1/(var(Estimate)),
    MSE = Bias^2 + var(Estimate),
  )
```

```r
E.Estimate.Summary |>
  knitr::kable(digits = 2)
```

Table 4: Exponential-Variance-Estimates.

Estimator	Bias	Mean	Precision	MSE
average.squared.distance	-0.12	1.88	0.77	1.30
sample.variance	-0.02	1.98	0.70	1.43

2.2 Simulation over various sample sizes

Consider the effect of the sample size have on these results. Provide a graphical and numerical summary of what happens as n increases – say $n = 20$, $n = 50$, $n = 100$, and $n = 500$. Does the result in your initial simulation hold across samples? Or, do things change as the sample size changes? Ensure to set your randomization seed.

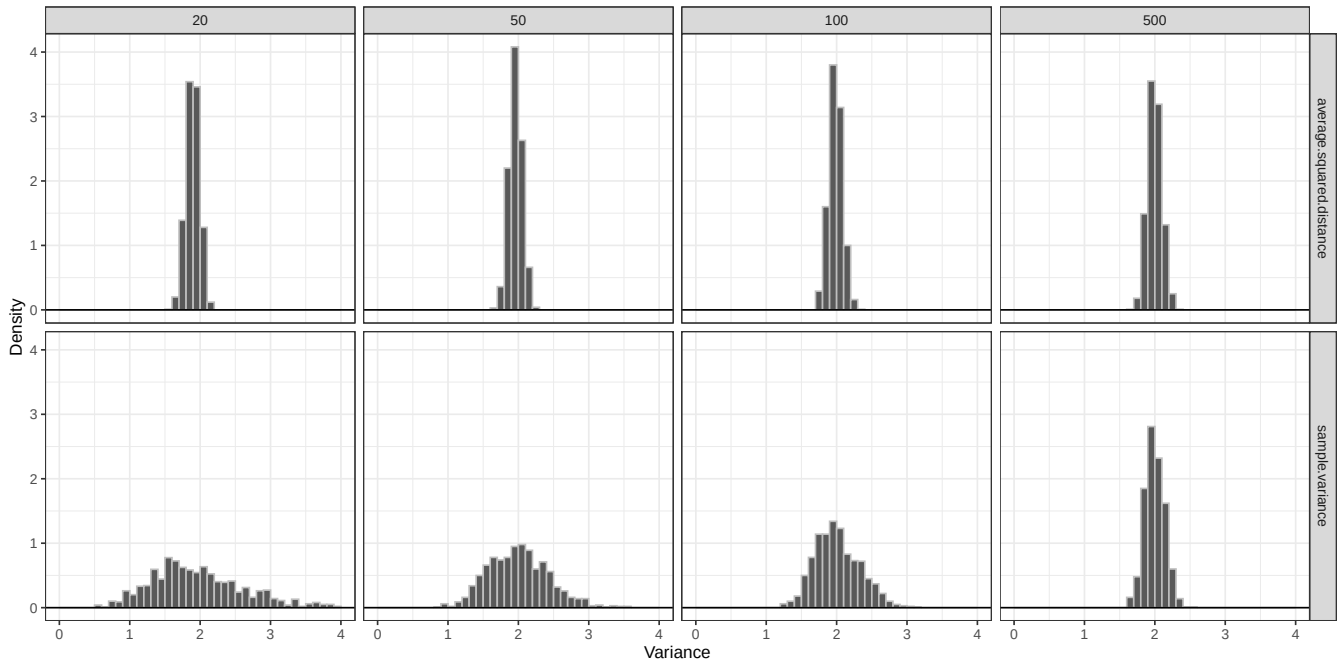
Solution

```r
set.seed(7272)

#Poisson Distribution
  psv = function(n) {
    psv_vals = numeric(1000)
    for (i in 1:1000) {
      x = rpois(n, 2)
      psv_vals[i] = var(x)
    }

    return(psv_vals)
  }

  pad = function(n) {
    pad_vals = numeric(1000)
    for (i in 1:1000) {
      x = rpois(n, 2)
      pad_vals[i] = ((n-1)/n)*var(x)
    }

    return(pad_vals)
  }

#Binomial Distribution
  bsv = function(n) {
    bsv_vals = numeric(1000)
    for (i in 1:1000) {
      x = rbinom(n, 50, (5-sqrt(21))/10)
      bsv_vals[i] = var(x)
    }

    return(bsv_vals)
  }

  bad = function(n) {
    bad_vals = numeric(1000)
    for (i in 1:1000) {
      x = rbinom(n, 50, (5-sqrt(21))/10)
      bad_vals[i] = ((n-1)/n)*var(x)
    }

    return(bad_vals)
  }

#Normal Distribution

  nsv = function(n) {
    nsv_vals = numeric(1000)
    for (i in 1:1000) {
      x =  rnorm(n, 0, sqrt(2))
      nsv_vals[i] = var(x)
    }

    return(nsv_vals)
  }
```

```r

  nad = function(n) {
    nad_vals = numeric(1000)
    for (i in 1:1000) {
      x =  rnorm(n, 0, sqrt(2))
      nad_vals[i] = ((n-1)/n)*var(x)
    }

    return(nad_vals)
  }

#Exponential Distribution
  esv = function(n) {
    esv_vals = numeric(1000)
    for (i in 1:1000) {
      x =  rexp(n, 1/(sqrt(2)))
      esv_vals[i] = var(x)
    }

    return(esv_vals)
  }

  ead = function(n) {
    ead_vals = numeric(1000)
    for (i in 1:1000) {
      x =  rexp(n, 1/(sqrt(2)))
      ead_vals[i] = ((n-1)/n)*var(x)
    }

    return(ead_vals)
  }
```

```r
n_values <- c(20, 50, 100, 500)

ggdat.p <- lapply(n_values, function(n) {
  tibble(
    n = n,
    sample.variance = psv(n),
    average.squared.distance = pad(n)
  )
}) |>
  bind_rows() |>
  pivot_longer(
    cols = c(sample.variance, average.squared.distance),
    names_to = "Estimator",
    values_to = "Estimate"
  )

ggplot(ggdat.p) +
  geom_histogram(
    aes(x = Estimate, y = after_stat(density)),
    breaks = seq(0, 4, 0.1),
    color = "grey"
  ) +
  geom_hline(yintercept = 0) +
  theme_bw() +
  labs(x = "Variance", y = "Density") +
  facet_grid(Estimator ~ n)
```

Figure 5: Poisson Variance Estimation.




```
P.Estimate.Summary = ggdat.p|>
  group_by(n, Estimator) |>
  summarize(
    Bias = mean(Estimate) - 2,
    Mean = mean(Estimate),
    Variance = var(Estimate),
    Precision = 1/(var(Estimate)),
    MSE = Bias^2 + var(Estimate),
  )
```

```
P.Estimate.Summary |>
  knitr::kable(digits = 2)
```

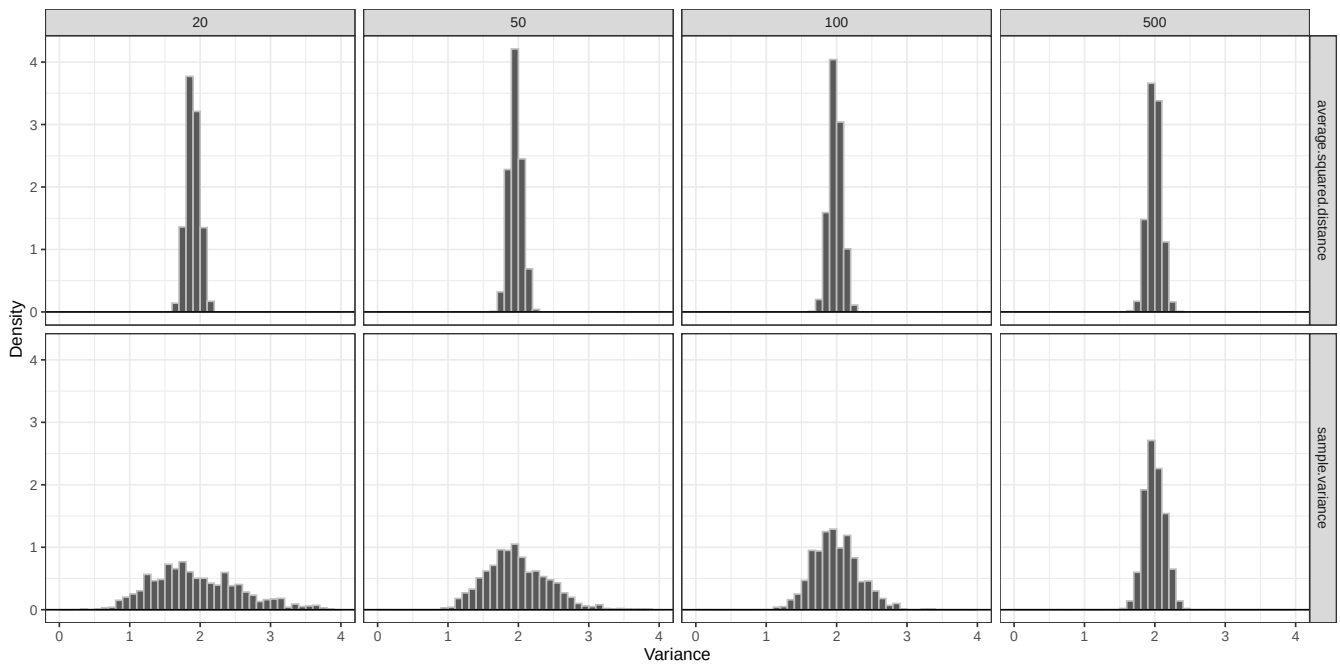
Table 5: Poisson-Variance-Estimates.

n	Estimator	Bias	Mean	Variance	Precision	MSE
20	average.squared.distance	-0.10	1.90	0.01	112.84	0.02
20	sample.variance	-0.02	1.98	0.48	2.10	0.48
50	average.squared.distance	-0.04	1.96	0.01	115.19	0.01
50	sample.variance	0.00	2.00	0.19	5.39	0.19
100	average.squared.distance	-0.02	1.98	0.01	108.72	0.01
100	sample.variance	0.01	2.01	0.10	9.72	0.10
500	average.squared.distance	0.00	2.00	0.01	99.13	0.01
500	sample.variance	0.00	2.00	0.02	52.74	0.02

```
n_values = c(20, 50, 100, 500)

ggdat.b = lapply(n_values, function(n) {
  tibble(
    n = n,
    sample.variance = bsv(n),
    average.squared.distance = bad(n)
  )
}) |>
  bind_rows() |>
```

```r
  pivot_longer(
    cols = c(sample.variance, average.squared.distance),
    names_to = "Estimator",
    values_to = "Estimate"
  )

ggplot(ggdat.b) +
  geom_histogram(
    aes(x = Estimate, y = after_stat(density)),
    breaks = seq(0, 4, 0.1),
    color = "grey"
  ) +
  geom_hline(yintercept = 0) +
  theme_bw() +
  labs(x = "Variance", y = "Density") +
  facet_grid(Estimator ~ n)
```

Figure 6: Binomial Variance Estimation.




```r
B.Estimate.Summary = ggdat.b|>
  group_by(n, Estimator) |>
  summarize(
    Bias = mean(Estimate) - 2,
    Mean = mean(Estimate),
    Variance = var(Estimate),
    Precision = 1/(var(Estimate)),
    MSE = Bias^2 + var(Estimate),
  )
```

```r
B.Estimate.Summary |>
  knitr::kable(digits = 2)
```

Table 6: Binomial-Variance-Estimates.

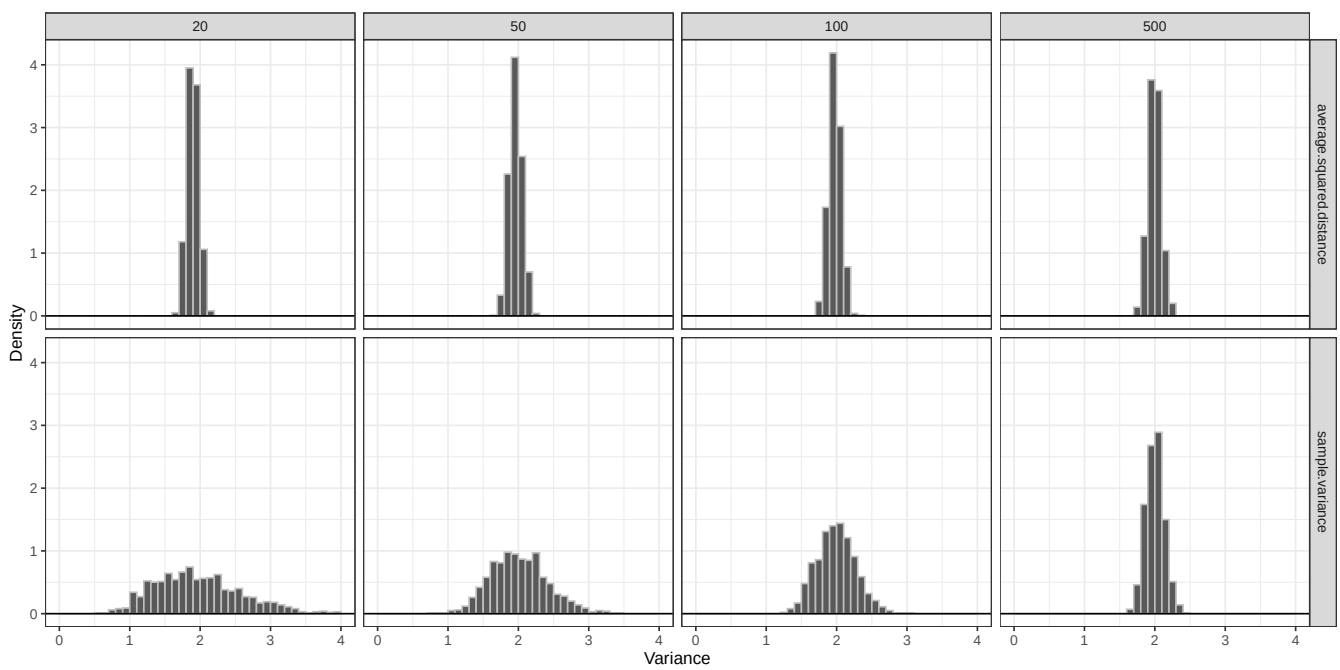
n	Estimator	Bias	Mean	Variance	Precision	MSE
20	average.squared.distance	-0.10	1.90	0.01	111.01	0.02
20	sample.variance	-0.03	1.97	0.46	2.15	0.47
50	average.squared.distance	-0.04	1.96	0.01	120.99	0.01
50	sample.variance	-0.02	1.98	0.20	5.02	0.20

Table 6: Binomial-Variance-Estimates.

n	Estimator	Bias	Mean	Variance	Precision	MSE
100	average.squared.distance	-0.02	1.98	0.01	112.35	0.01
100	sample.variance	-0.01	1.99	0.10	9.56	0.10
500	average.squared.distance	-0.01	1.99	0.01	107.63	0.01
500	sample.variance	-0.01	1.99	0.02	50.20	0.02

```r
n_values = c(20, 50, 100, 500)

ggdat.n = lapply(n_values, function(n) {
  tibble(
    n = n,
    sample.variance = nsv(n),
    average.squared.distance = nad(n)
  )
}) |>
  bind_rows() |>
  pivot_longer(
    cols = c(sample.variance, average.squared.distance),
    names_to = "Estimator",
    values_to = "Estimate"
  )

ggplot(ggdat.n) +
  geom_histogram(
    aes(x = Estimate, y = after_stat(density)),
    breaks = seq(0, 4, 0.1),
    color = "grey"
  ) +
  geom_hline(yintercept = 0) +
  theme_bw() +
  labs(x = "Variance", y = "Density") +
  facet_grid(Estimator ~ n)
```

Figure 7: Normal Variance Estimation.




```r
N.Estimate.Summary = ggdat.n|>
  group_by(n, Estimator) |>
  summarize(
    Bias = mean(Estimate) - 2,
    Mean = mean(Estimate),
```

```r
    Variance = var(Estimate),
    Precision = 1/(var(Estimate)),
    MSE = Bias^2 + var(Estimate),
  )
```

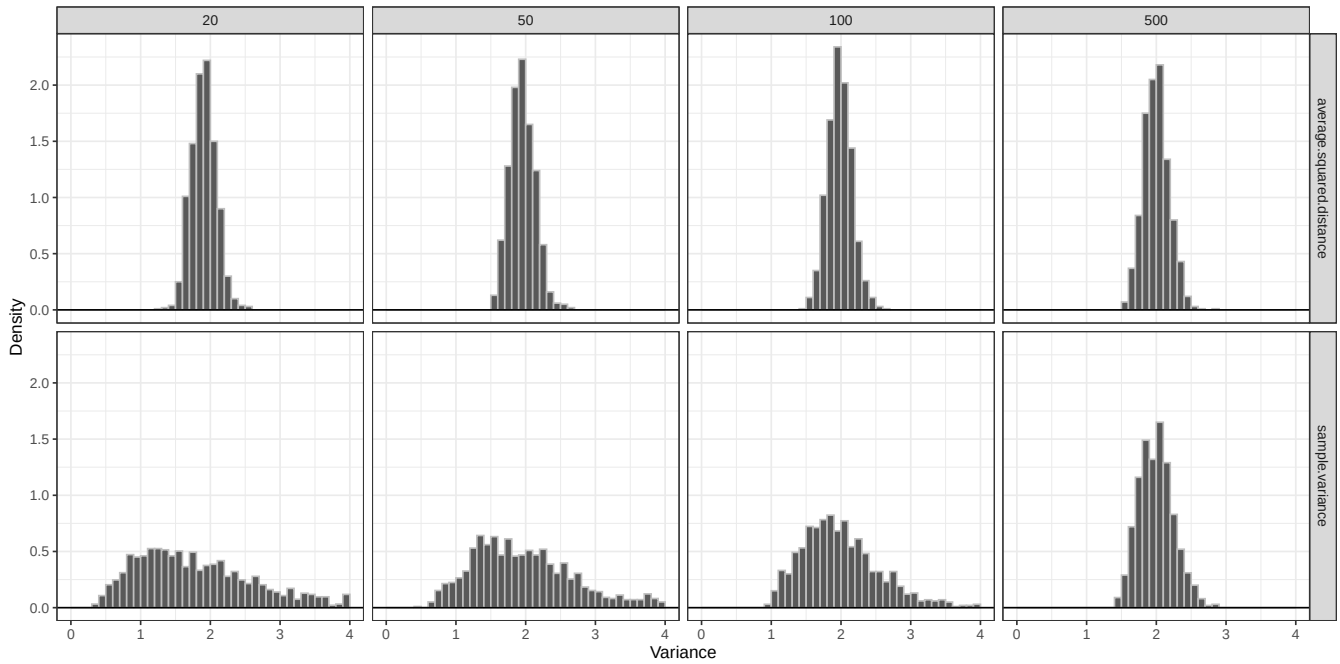
```r
N.Estimate.Summary |>
  knitr::kable(digits = 2)
```

Table 7: Normal-Variance-Estimates.

n	Estimator	Bias	Mean	Variance	Precision	MSE
20	average.squared.distance	-0.10	1.90	0.01	141.04	0.02
20	sample.variance	-0.02	1.98	0.40	2.48	0.40
50	average.squared.distance	-0.04	1.96	0.01	116.30	0.01
50	sample.variance	0.01	2.01	0.17	5.96	0.17
100	average.squared.distance	-0.02	1.98	0.01	128.50	0.01
100	sample.variance	0.00	2.00	0.08	13.27	0.08
500	average.squared.distance	0.00	2.00	0.01	118.10	0.01
500	sample.variance	0.00	2.00	0.02	59.21	0.02

```r
n_values = c(20, 50, 100, 500)

ggdat.e = lapply(n_values, function(n) {
  tibble(
    n = n,
    sample.variance = esv(n),
    average.squared.distance = ead(n)
  )
}) |>
  bind_rows() |>
  pivot_longer(
    cols = c(sample.variance, average.squared.distance),
    names_to = "Estimator",
    values_to = "Estimate"
  )

ggplot(ggdat.e) +
  geom_histogram(
    aes(x = Estimate, y = after_stat(density)),
    breaks = seq(0, 4, 0.1),
    color = "grey"
  ) +
  geom_hline(yintercept = 0) +
  theme_bw() +
  labs(x = "Variance", y = "Density") +
  facet_grid(Estimator ~ n)
```

Figure 8: Exponential Variance Estimation.




```r
E.Estimate.Summary = ggdat.e|>
  group_by(n, Estimator) |>
  summarize(
    Bias = mean(Estimate) - 2,
    Mean = mean(Estimate),
    Variance = var(Estimate),
    Precision = 1/(var(Estimate)),
    MSE = Bias^2 + var(Estimate),
  )
```

```r
E.Estimate.Summary |>
  knitr::kable(digits = 2)
```

Table 8: Exponential-Variance-Estimates.

n	Estimator	Bias	Mean	Variance	Precision	MSE
20	average.squared.distance	-0.09	1.91	0.03	31.74	0.04
20	sample.variance	0.01	2.01	1.42	0.70	1.42
50	average.squared.distance	-0.05	1.95	0.03	30.74	0.03
50	sample.variance	0.01	2.01	0.66	1.52	0.66
100	average.squared.distance	-0.02	1.98	0.03	32.10	0.03
100	sample.variance	0.00	2.00	0.33	3.06	0.33
500	average.squared.distance	0.00	2.00	0.03	29.66	0.03
500	sample.variance	0.00	2.00	0.06	16.24	0.06

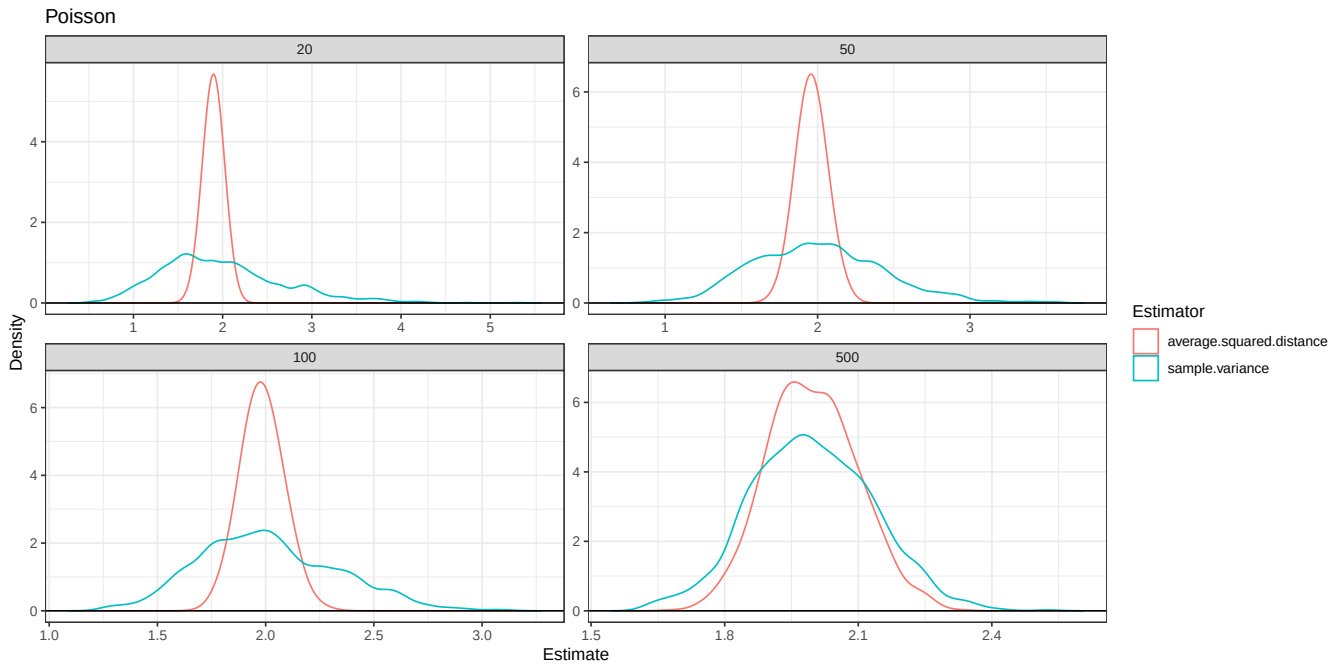
2.3 Simulation

You have written code to draw a random sample of various sizes from each distribution, and to calculate and store the calculated estimates. Rework your code to produce a density plot of the s_{n-1}^2 and s_n^2. Do you have a guess about their distribution? Ensure to set your randomization seed.

```r
set.seed(7272)
library(ggridges)
```

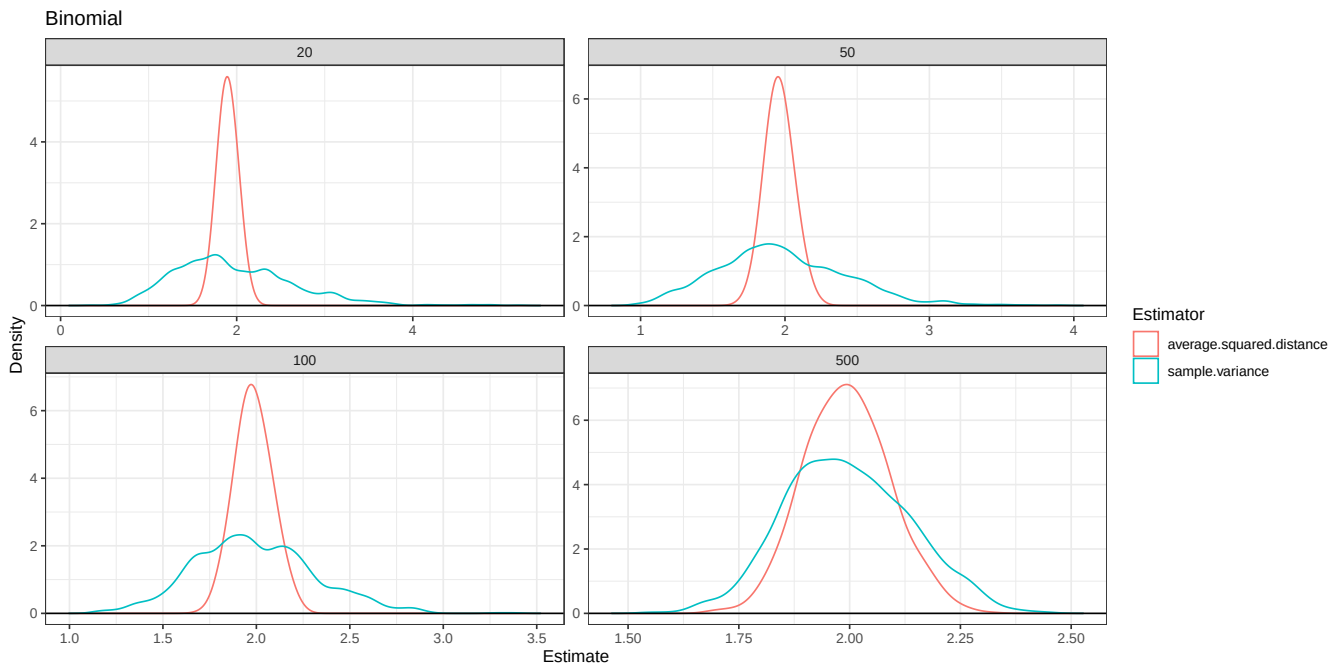
```r

ggplot(ggdat.p) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Poisson") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 9: Poisson Density Plot Variance Estimation.



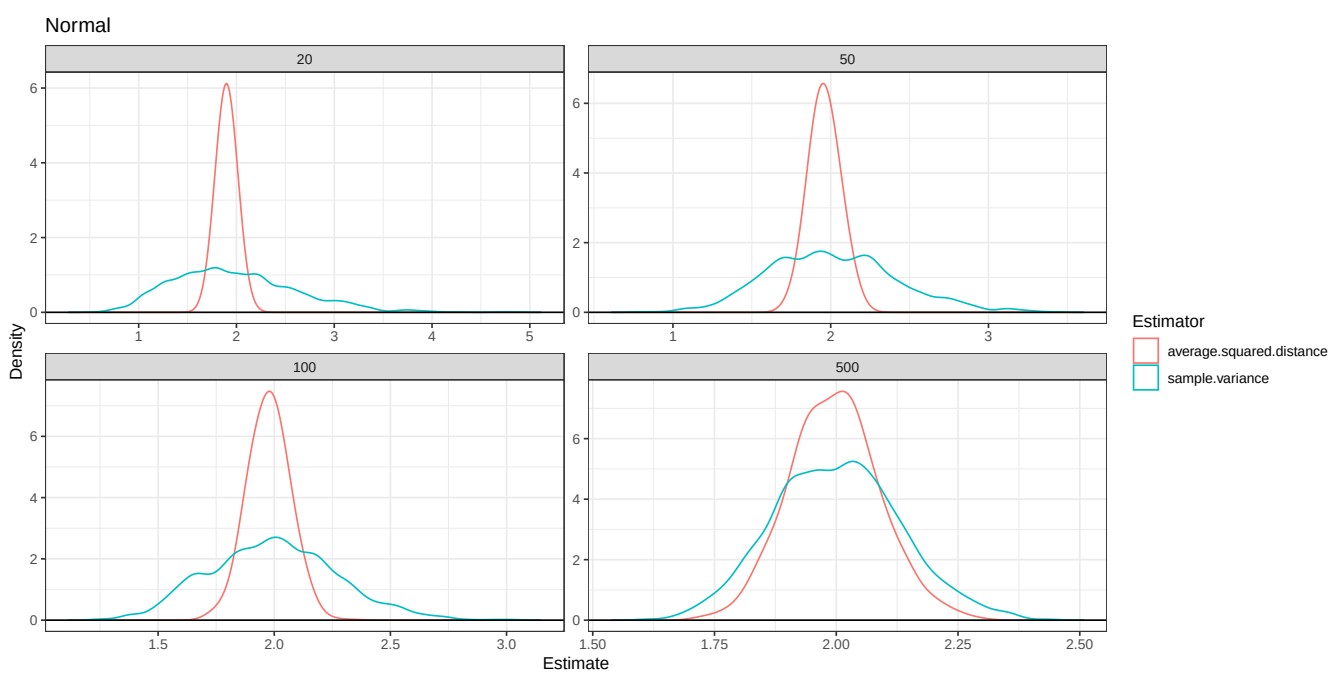

```r
set.seed(7272)

ggplot(ggdat.b) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Binomial") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 10: Binomial Density Plot Variance Estimation.



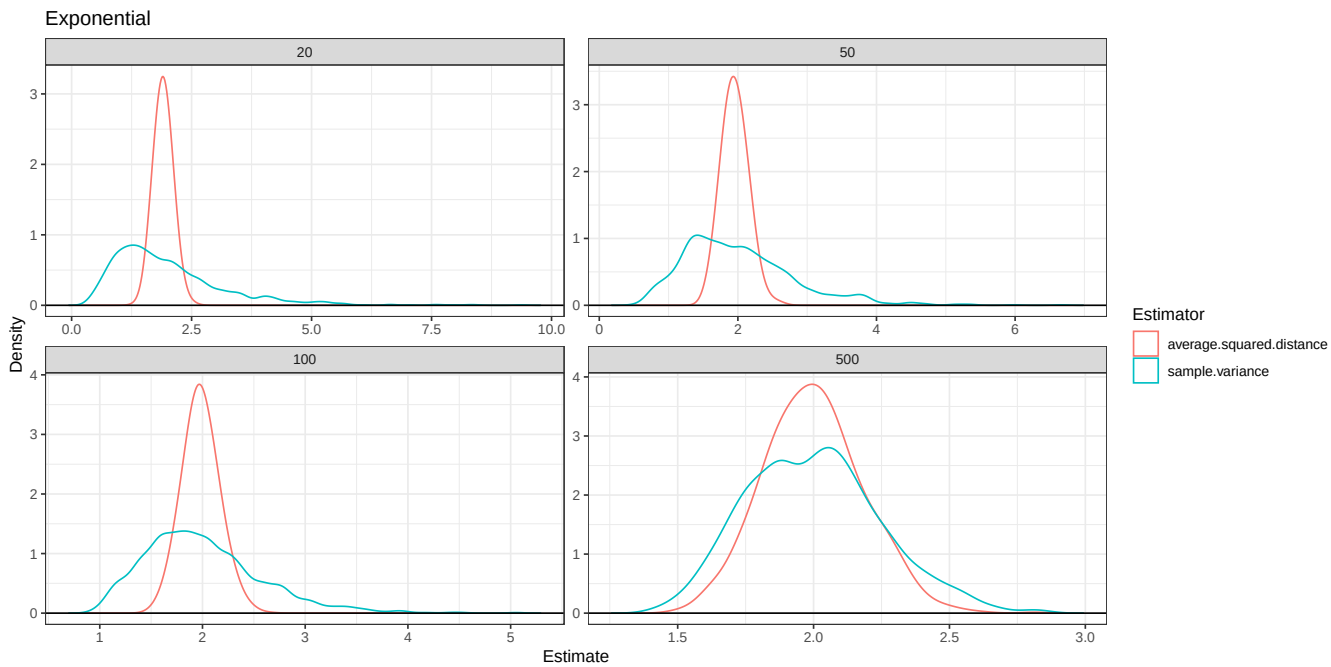

```
set.seed(7272)

ggplot(ggdat.n) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Normal") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 11: Normal Density Plot Variance Estimation.



```r
set.seed(7272)

ggplot(ggdat.e) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Exponential") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 12: Exponential Density Plot Variance Estimation.



2.4 Bootstrap

Now, take *one* sample from each distribution. Instead of drawing a random sample at each iteration, sample *with replacement* from the one sample. Produce a density plot of the s^2_{n-1} and s^2_n. Do we get roughly the same information as we do in the full simulation? Try this over several seeds – what do you notice? Ensure to set your randomization seed.

Solution

With the bootstrap sample, as with having multiple smaples, the sampling distributions of variance typically follow a Gaussian distribution, becoming more normal and symmetric as the number of replacements increase (similar to if the number of samples increased). However, using bootstrapping can lead to less accurate data, as these distributions were centered at values other than the true population variance of two. Thus, it is important that only a representative sample is bootstrapped, which is often incredibly difficult. Additionally, as the number of replacements increased, the respective distributions of average squared distance away from the mean and the sample variance became very similar.

With Seed(7272)

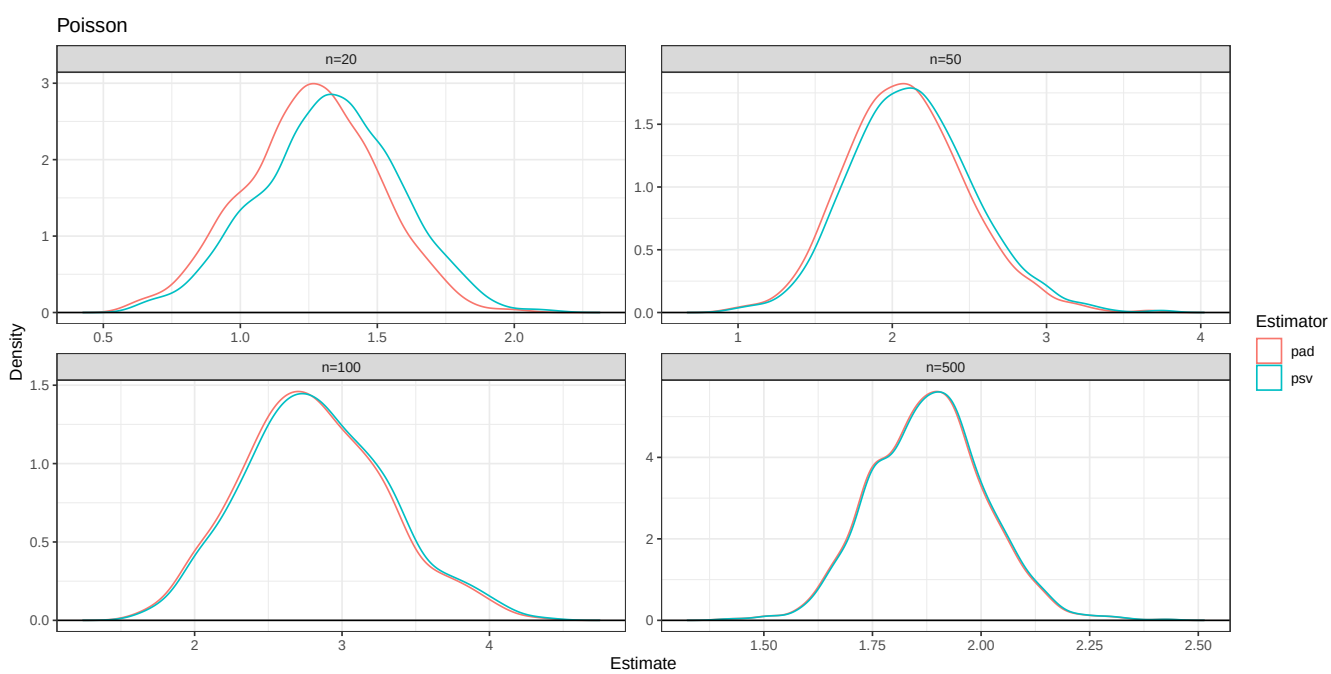
```r
set.seed(7272)

n.values = c(20, 50, 100, 500)

# poisson
 pois.sample = list()
 psv_b = list()
```

```r
 pad_b = list()
 psv = rep(NA, 1000)
 pad = rep(NA, 1000)

for(n in n.values){
# take 1 random sample of different sizes
  pois.sample[[as.character(n)]] <- rpois(n = n, lambda = 2)
  for(i in 1:1000){
    curr.sample = sample(pois.sample[[as.character(n)]], size = n, replace = TRUE)
    x_bar = mean(curr.sample)
    psv[i] = sum((curr.sample - x_bar)^2) / (n - 1)
    pad[i] = sum((curr.sample - x_bar)^2) / n
  }
  psv_b[[as.character(n)]] = psv
  pad_b[[as.character(n)]] = pad
}

# binomial
 binom.sample = list()
 bsv_b = list()
 bad_b = list()
 bsv = rep(NA, 1000)
 bad = rep(NA, 1000)

for(n in n.values){
# take 1 random sample of different sizes
  binom.sample[[as.character(n)]] = rbinom(n, 50, (5-sqrt(21))/10)
  for(i in 1:1000){
    curr.sample = sample(binom.sample[[as.character(n)]], size = n, replace = TRUE)
    x_bar = mean(curr.sample)
    bsv[i] = sum((curr.sample - x_bar)^2) / (n - 1)
    bad[i] = sum((curr.sample - x_bar)^2) / n
  }
  bsv_b[[as.character(n)]] = bsv
  bad_b[[as.character(n)]] = bad
}

# normal
 norm.sample = list()
 nsv_b = list()
 nad_b = list()
 nsv = rep(NA, 1000)
 nad = rep(NA, 1000)

for(n in n.values){
# take 1 random sample of different sizes
  norm.sample[[as.character(n)]] = rnorm(n, 0, sqrt(2))
  for(i in 1:1000){
    curr.sample = sample(norm.sample[[as.character(n)]], size = n, replace = TRUE)
    x_bar = mean(curr.sample)
    nsv[i] = sum((curr.sample - x_bar)^2) / (n - 1)
    nad[i] = sum((curr.sample - x_bar)^2) / n
  }
  nsv_b[[as.character(n)]] = nsv
  nad_b[[as.character(n)]] = nad
}
```

```r

# exponential
 exp.sample = list()
 esv_b = list()
 ead_b = list()
 esv = rep(NA, 1000)
 ead = rep(NA, 1000)

for(n in n.values){
# take 1 random sample of different sizes
  exp.sample[[as.character(n)]] = rexp(n, 1/(sqrt(2)))
  for(i in 1:1000){
    curr.sample = sample(exp.sample[[as.character(n)]], size = n, replace = TRUE)
    x_bar = mean(curr.sample)
    esv[i] = sum((curr.sample - x_bar)^2) / (n - 1)
    ead[i] = sum((curr.sample - x_bar)^2) / n
  }
  esv_b[[as.character(n)]] = esv
  ead_b[[as.character(n)]] = ead
}
```

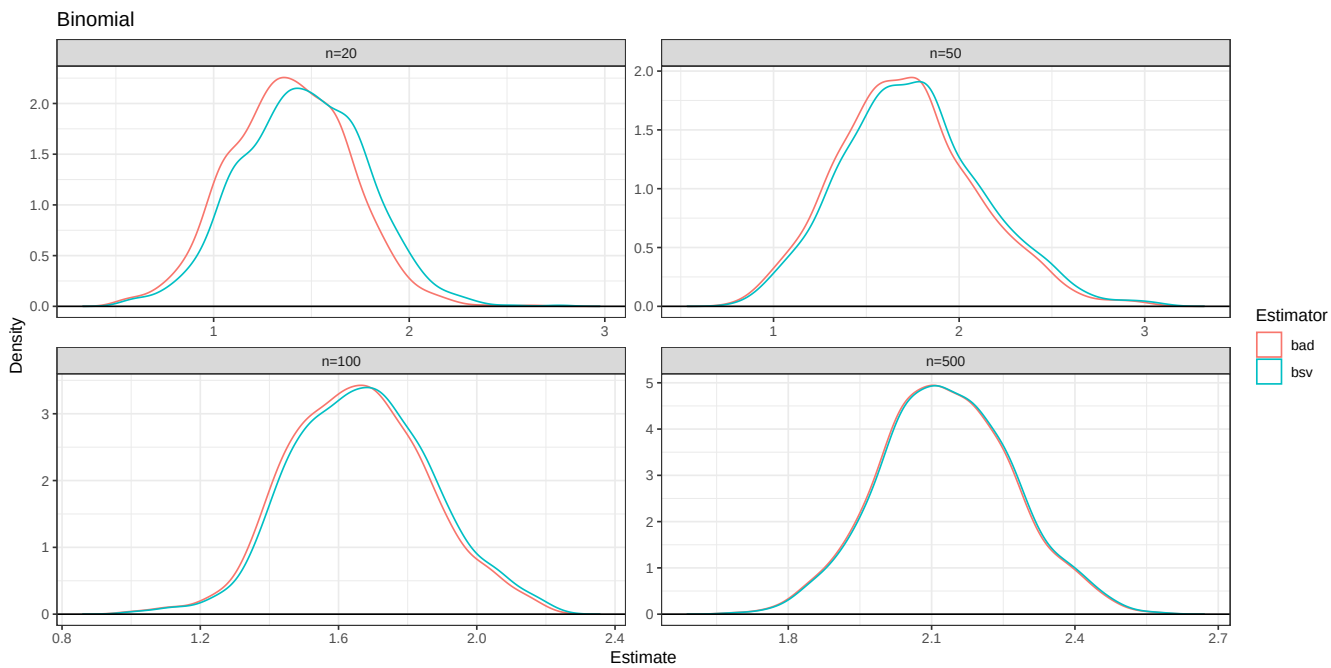
```r
ggdat.pb <- tibble(
Estimator = rep(c("psv", "pad"), each = 1000, times = 4),
n = factor(rep(c("n=20", "n=50", "n=100", "n=500"), each = 2000),
levels = c("n=20", "n=50", "n=100", "n=500")),
Estimate = c(psv_b[["20"]], pad_b[["20"]],
psv_b[["50"]], pad_b[["50"]],
psv_b[["100"]], pad_b[["100"]],
psv_b[["500"]], pad_b[["500"]])
)

ggplot(ggdat.pb) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Poisson") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 13: Poisson Density Plot Variance Estimation with Bootstrapping.



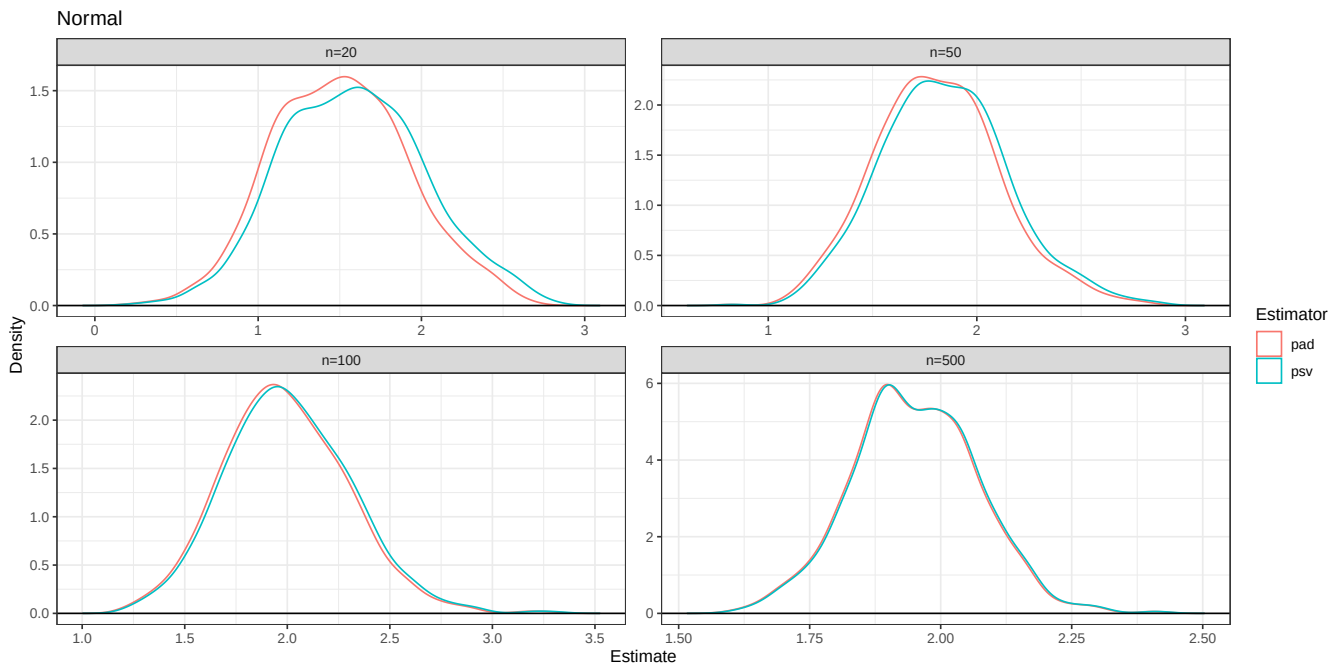
```r
ggdat.bb <- tibble(
Estimator = rep(c("bsv", "bad"), each = 1000, times = 4),
n = factor(rep(c("n=20", "n=50", "n=100", "n=500"), each = 2000),
levels = c("n=20", "n=50", "n=100", "n=500")),
Estimate = c(bsv_b[["20"]], bad_b[["20"]],
bsv_b[["50"]], bad_b[["50"]],
bsv_b[["100"]], bad_b[["100"]],
bsv_b[["500"]], bad_b[["500"]])
)

ggplot(ggdat.bb) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Binomial") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 14: Binomial Density Plot Variance Estimation with Bootstrapping.



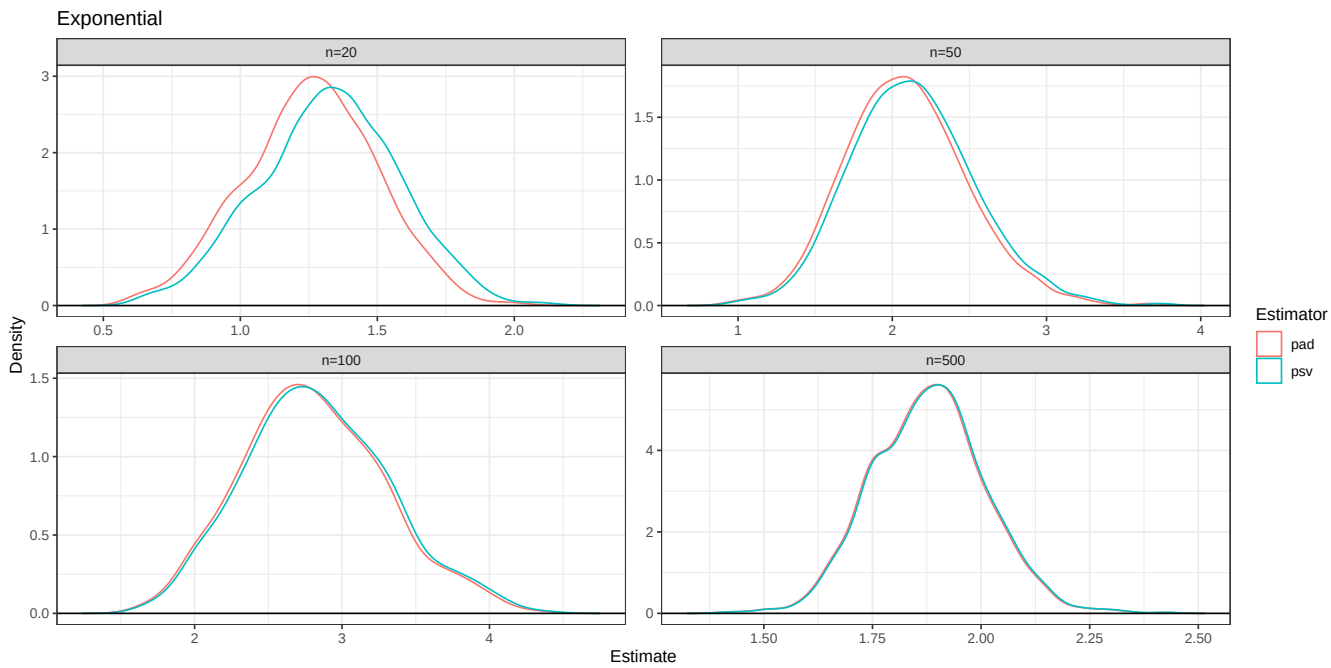

```r
ggdat.nb <- tibble(
Estimator = rep(c("psv", "pad"), each = 1000, times = 4),
n = factor(rep(c("n=20", "n=50", "n=100", "n=500"), each = 2000),
levels = c("n=20", "n=50", "n=100", "n=500")),
Estimate = c(nsv_b[["20"]], nad_b[["20"]],
nsv_b[["50"]], nad_b[["50"]],
nsv_b[["100"]], nad_b[["100"]],
nsv_b[["500"]], nad_b[["500"]])
)

ggplot(ggdat.nb) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Normal") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 15: Normal Density Plot Variance Estimation with Bootstrapping.




```
ggdat.eb <- tibble(
Estimator = rep(c("psv", "pad"), each = 1000, times = 4),
n = factor(rep(c("n=20", "n=50", "n=100", "n=500"), each = 2000),
levels = c("n=20", "n=50", "n=100", "n=500")),
Estimate = c(psv_b[["20"]], pad_b[["20"]],
psv_b[["50"]], pad_b[["50"]],
psv_b[["100"]], pad_b[["100"]],
psv_b[["500"]], pad_b[["500"]])
)

ggplot(ggdat.eb) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Exponential") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 16: Exponential Density Plot Variance Estimation with Bootstrapping.



With seed(1325)

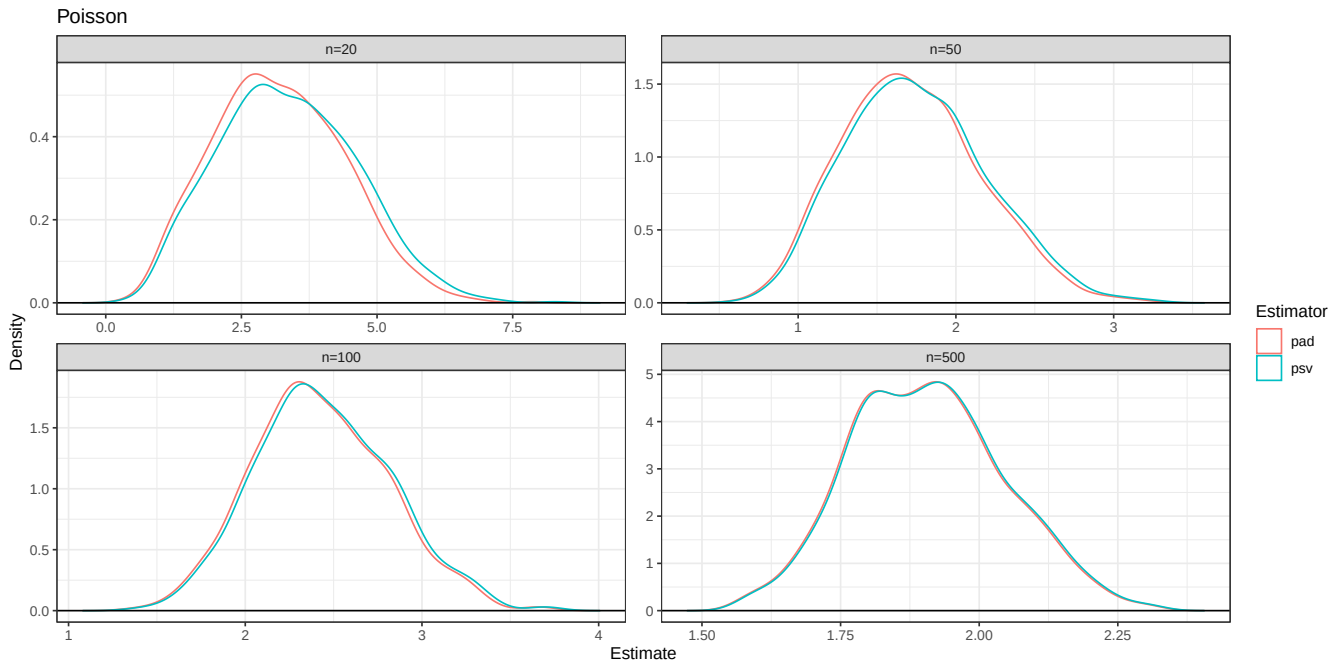
```
set.seed(1325)

n.values = c(20, 50, 100, 500)

# poisson
 pois.sample = list()
 psv_b = list()
 pad_b = list()
 psv = rep(NA, 1000)
 pad = rep(NA, 1000)

for(n in n.values){
# take 1 random sample of different sizes
  pois.sample[[as.character(n)]] <- rpois(n = n, lambda = 2)
  for(i in 1:1000){
    curr.sample = sample(pois.sample[[as.character(n)]], size = n, replace = TRUE)
    x_bar = mean(curr.sample)
    psv[i] = sum((curr.sample - x_bar)^2) / (n - 1)
    pad[i] = sum((curr.sample - x_bar)^2) / n
  }
  psv_b[[as.character(n)]] = psv
  pad_b[[as.character(n)]] = pad
}

# binomial
 binom.sample = list()
 bsv_b = list()
 bad_b = list()
 bsv = rep(NA, 1000)
 bad = rep(NA, 1000)

```

```r
for(n in n.values){
# take 1 random sample of different sizes
  binom.sample[[as.character(n)]] = rbinom(n, 50, (5-sqrt(21))/10)
  for(i in 1:1000){
    curr.sample = sample(binom.sample[[as.character(n)]], size = n, replace = TRUE)
    x_bar = mean(curr.sample)
    bsv[i] = sum((curr.sample - x_bar)^2) / (n - 1)
    bad[i] = sum((curr.sample - x_bar)^2) / n
  }
  bsv_b[[as.character(n)]] = bsv
  bad_b[[as.character(n)]] = bad
}

# normal
 norm.sample = list()
 nsv_b = list()
 nad_b = list()
 nsv = rep(NA, 1000)
 nad = rep(NA, 1000)

for(n in n.values){
# take 1 random sample of different sizes
  norm.sample[[as.character(n)]] = rnorm(n, 0, sqrt(2))
  for(i in 1:1000){
    curr.sample = sample(norm.sample[[as.character(n)]], size = n, replace = TRUE)
    x_bar = mean(curr.sample)
    nsv[i] = sum((curr.sample - x_bar)^2) / (n - 1)
    nad[i] = sum((curr.sample - x_bar)^2) / n
  }
  nsv_b[[as.character(n)]] = nsv
  nad_b[[as.character(n)]] = nad
}

# exponential
 exp.sample = list()
 esv_b = list()
 ead_b = list()
 esv = rep(NA, 1000)
 ead = rep(NA, 1000)

for(n in n.values){
# take 1 random sample of different sizes
  exp.sample[[as.character(n)]] = rexp(n, 1/(sqrt(2)))
  for(i in 1:1000){
    curr.sample = sample(exp.sample[[as.character(n)]], size = n, replace = TRUE)
    x_bar = mean(curr.sample)
    esv[i] = sum((curr.sample - x_bar)^2) / (n - 1)
    ead[i] = sum((curr.sample - x_bar)^2) / n
  }
  esv_b[[as.character(n)]] = esv
  ead_b[[as.character(n)]] = ead
}
```

```r
ggdat.pb <- tibble(
Estimator = rep(c("psv", "pad"), each = 1000, times = 4),
n = factor(rep(c("n=20", "n=50", "n=100", "n=500"), each = 2000),
levels = c("n=20", "n=50", "n=100", "n=500")),
Estimate = c(psv_b[["20"]], pad_b[["20"]],
psv_b[["50"]], pad_b[["50"]],
```

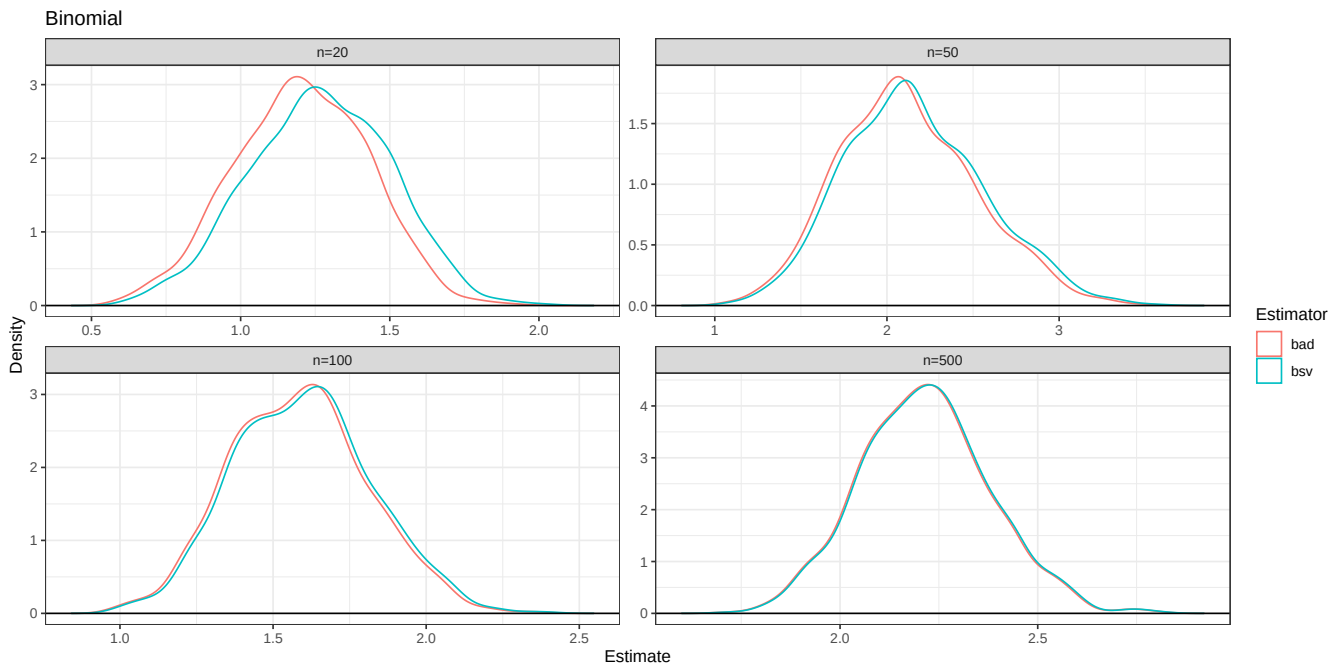
```
psv_b[["100"]], pad_b[["100"]],
psv_b[["500"]], pad_b[["500"]])
)

ggplot(ggdat.pb) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Poisson") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 17: Poisson Density Plot Variance Estimation with Bootstrapping.



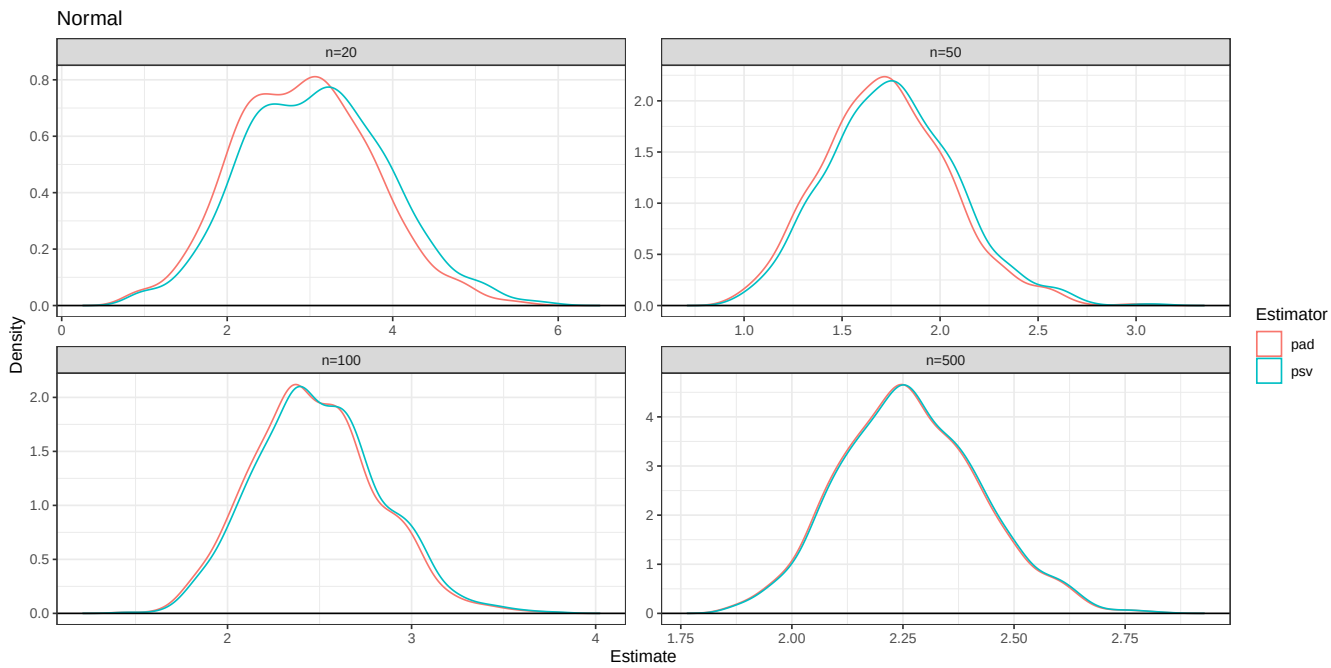

```
ggdat.bb <- tibble(
Estimator = rep(c("bsv", "bad"), each = 1000, times = 4),
n = factor(rep(c("n=20", "n=50", "n=100", "n=500"), each = 2000),
levels = c("n=20", "n=50", "n=100", "n=500")),
Estimate = c(bsv_b[["20"]], bad_b[["20"]],
bsv_b[["50"]], bad_b[["50"]],
bsv_b[["100"]], bad_b[["100"]],
bsv_b[["500"]], bad_b[["500"]])
)

ggplot(ggdat.bb) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Binomial") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 18: Binomial Density Plot Variance Estimation with Bootstrapping.



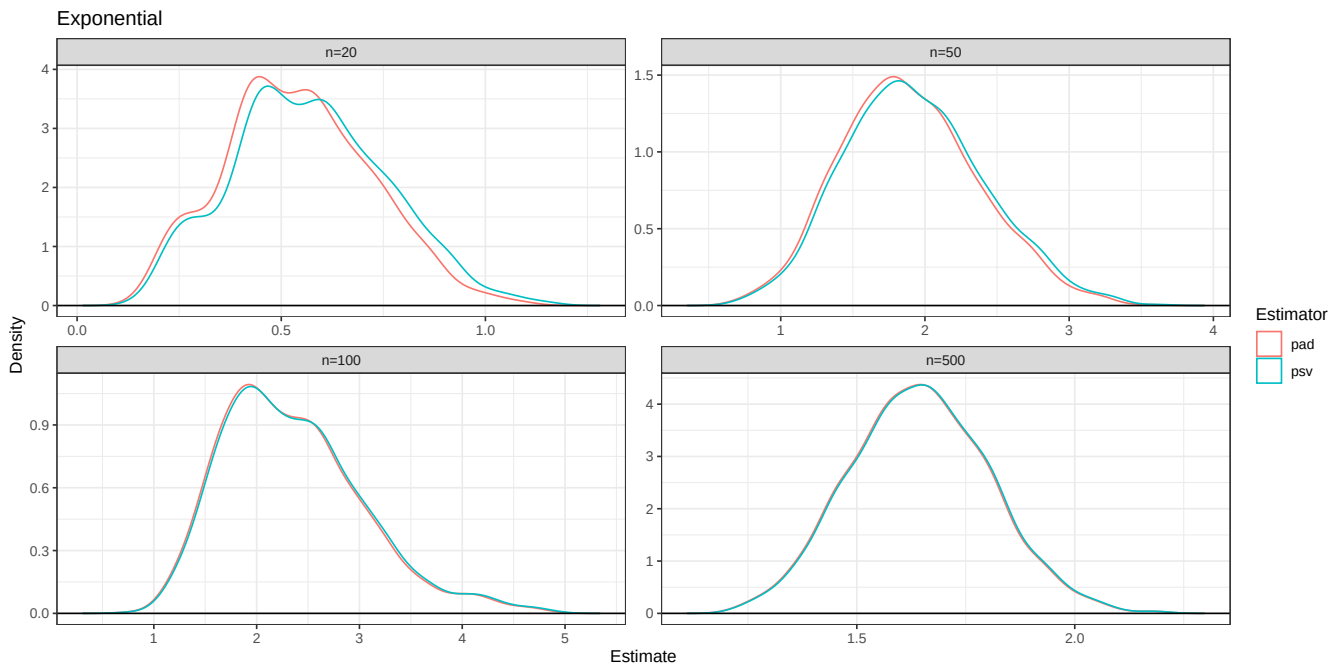

```r
ggdat.nb <- tibble(
Estimator = rep(c("psv", "pad"), each = 1000, times = 4),
n = factor(rep(c("n=20", "n=50", "n=100", "n=500"), each = 2000),
levels = c("n=20", "n=50", "n=100", "n=500")),
Estimate = c(nsv_b[["20"]], nad_b[["20"]],
nsv_b[["50"]], nad_b[["50"]],
nsv_b[["100"]], nad_b[["100"]],
nsv_b[["500"]], nad_b[["500"]])
)

ggplot(ggdat.nb) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Normal") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 19: Normal Density Plot Variance Estimation with Bootstrapping.




```
ggdat.eb <- tibble(
Estimator = rep(c("psv", "pad"), each = 1000, times = 4),
n = factor(rep(c("n=20", "n=50", "n=100", "n=500"), each = 2000),
levels = c("n=20", "n=50", "n=100", "n=500")),
Estimate = c(esv_b[["20"]], ead_b[["20"]],
esv_b[["50"]], ead_b[["50"]],
esv_b[["100"]], ead_b[["100"]],
esv_b[["500"]], ead_b[["500"]])
)

ggplot(ggdat.eb) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Exponential") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 20: Exponential Density Plot Variance Estimation with Bootstrapping.



With Seed(3434)

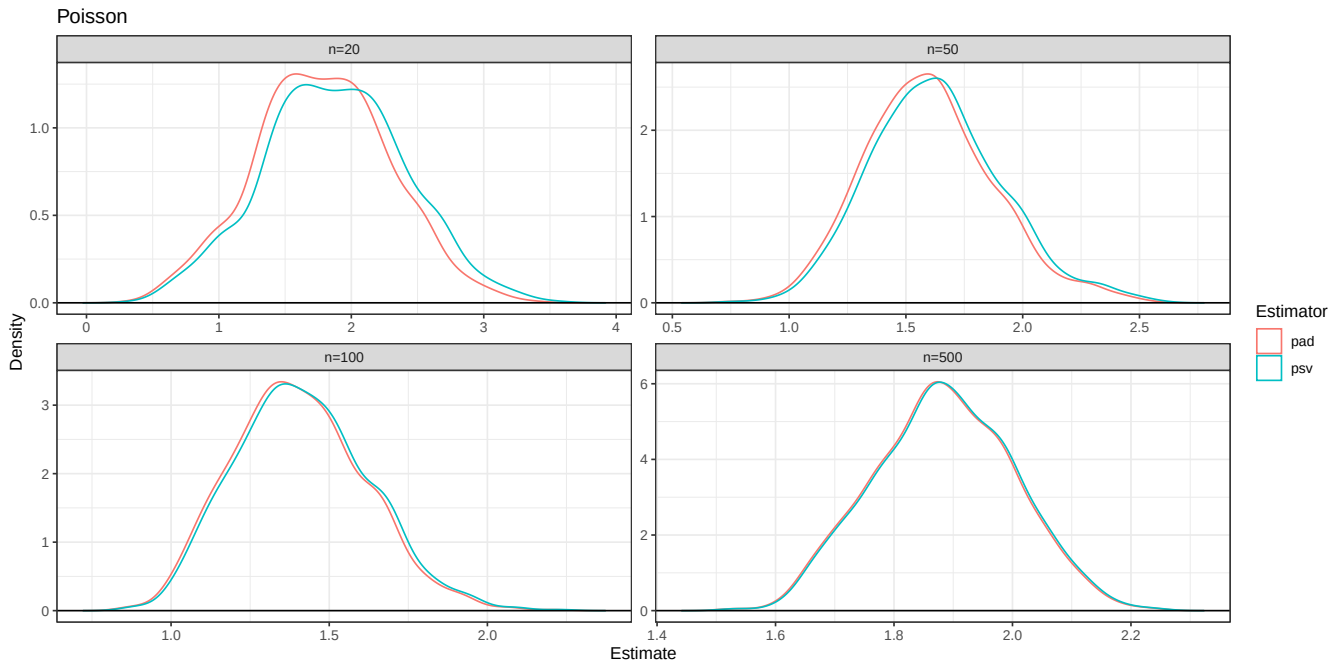
```
set.seed(3434)

n.values = c(20, 50, 100, 500)

# poisson
 pois.sample = list()
 psv_b = list()
 pad_b = list()
 psv = rep(NA, 1000)
 pad = rep(NA, 1000)

for(n in n.values){
# take 1 random sample of different sizes
  pois.sample[[as.character(n)]] <- rpois(n = n, lambda = 2)
  for(i in 1:1000){
    curr.sample = sample(pois.sample[[as.character(n)]], size = n, replace = TRUE)
    x_bar = mean(curr.sample)
    psv[i] = sum((curr.sample - x_bar)^2) / (n - 1)
    pad[i] = sum((curr.sample - x_bar)^2) / n
  }
  psv_b[[as.character(n)]] = psv
  pad_b[[as.character(n)]] = pad
}

# binomial
 binom.sample = list()
 bsv_b = list()
 bad_b = list()
 bsv = rep(NA, 1000)
 bad = rep(NA, 1000)

```

```r
for(n in n.values){
# take 1 random sample of different sizes
  binom.sample[[as.character(n)]] = rbinom(n, 50, (5-sqrt(21))/10)
  for(i in 1:1000){
    curr.sample = sample(binom.sample[[as.character(n)]], size = n, replace = TRUE)
    x_bar = mean(curr.sample)
    bsv[i] = sum((curr.sample - x_bar)^2) / (n - 1)
    bad[i] = sum((curr.sample - x_bar)^2) / n
  }
  bsv_b[[as.character(n)]] = bsv
  bad_b[[as.character(n)]] = bad
}

# normal
 norm.sample = list()
 nsv_b = list()
 nad_b = list()
 nsv = rep(NA, 1000)
 nad = rep(NA, 1000)

for(n in n.values){
# take 1 random sample of different sizes
  norm.sample[[as.character(n)]] = rnorm(n, 0, sqrt(2))
  for(i in 1:1000){
    curr.sample = sample(norm.sample[[as.character(n)]], size = n, replace = TRUE)
    x_bar = mean(curr.sample)
    nsv[i] = sum((curr.sample - x_bar)^2) / (n - 1)
    nad[i] = sum((curr.sample - x_bar)^2) / n
  }
  nsv_b[[as.character(n)]] = nsv
  nad_b[[as.character(n)]] = nad
}

# exponential
 exp.sample = list()
 esv_b = list()
 ead_b = list()
 esv = rep(NA, 1000)
 ead = rep(NA, 1000)

for(n in n.values){
# take 1 random sample of different sizes
  exp.sample[[as.character(n)]] = rexp(n, 1/(sqrt(2)))
  for(i in 1:1000){
    curr.sample = sample(exp.sample[[as.character(n)]], size = n, replace = TRUE)
    x_bar = mean(curr.sample)
    esv[i] = sum((curr.sample - x_bar)^2) / (n - 1)
    ead[i] = sum((curr.sample - x_bar)^2) / n
  }
  esv_b[[as.character(n)]] = esv
  ead_b[[as.character(n)]] = ead
}
```

```r
ggdat.pb <- tibble(
Estimator = rep(c("psv", "pad"), each = 1000, times = 4),
n = factor(rep(c("n=20", "n=50", "n=100", "n=500"), each = 2000),
levels = c("n=20", "n=50", "n=100", "n=500")),
Estimate = c(psv_b[["20"]], pad_b[["20"]],
psv_b[["50"]], pad_b[["50"]],
```

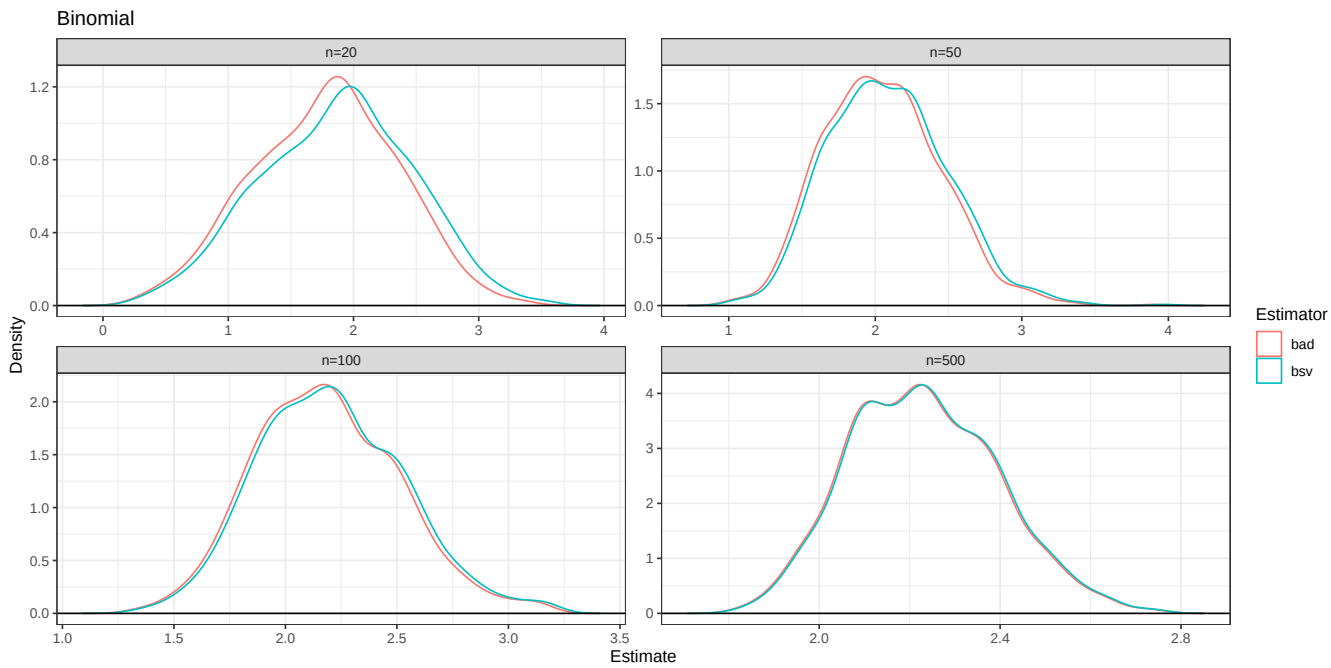
```
psv_b[["100"]], pad_b[["100"]],
psv_b[["500"]], pad_b[["500"]])
)

ggplot(ggdat.pb) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Poisson") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 21: Poisson Density Plot Variance Estimation with Bootstrapping.



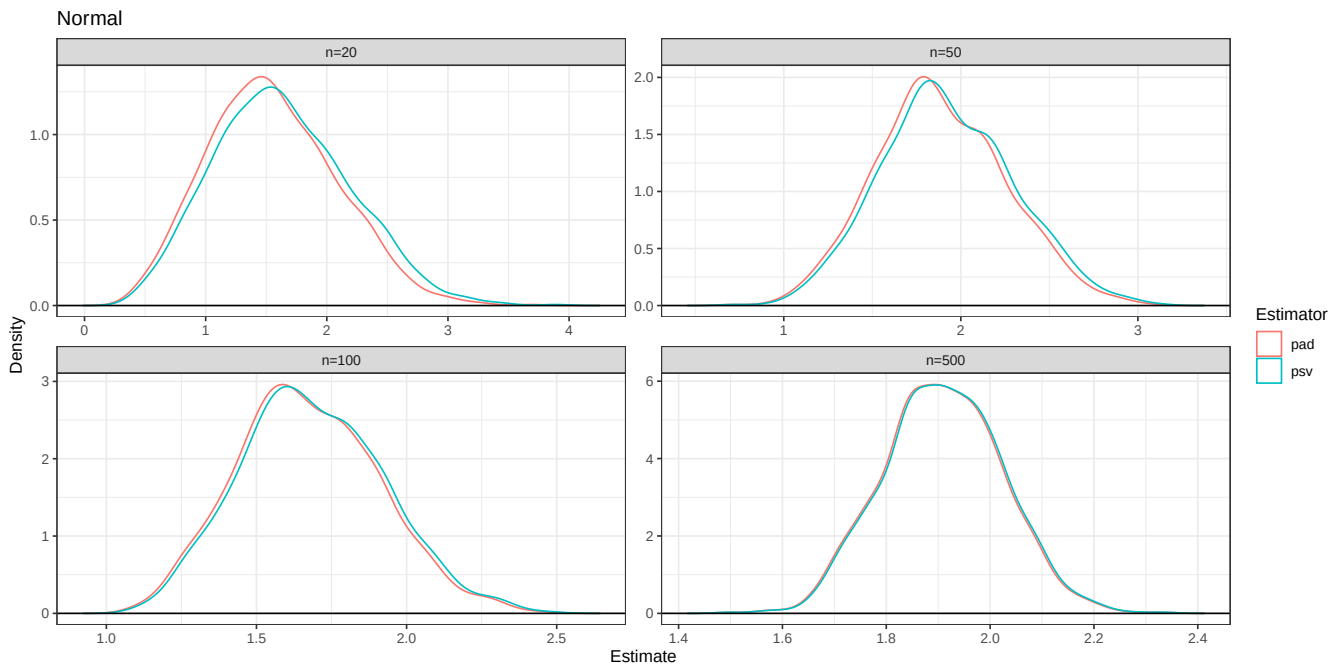

```
ggdat.bb <- tibble(
Estimator = rep(c("bsv", "bad"), each = 1000, times = 4),
n = factor(rep(c("n=20", "n=50", "n=100", "n=500"), each = 2000),
levels = c("n=20", "n=50", "n=100", "n=500")),
Estimate = c(bsv_b[["20"]], bad_b[["20"]],
bsv_b[["50"]], bad_b[["50"]],
bsv_b[["100"]], bad_b[["100"]],
bsv_b[["500"]], bad_b[["500"]])
)

ggplot(ggdat.bb) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Binomial") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 22: Binomial Density Plot Variance Estimation with Bootstrapping.



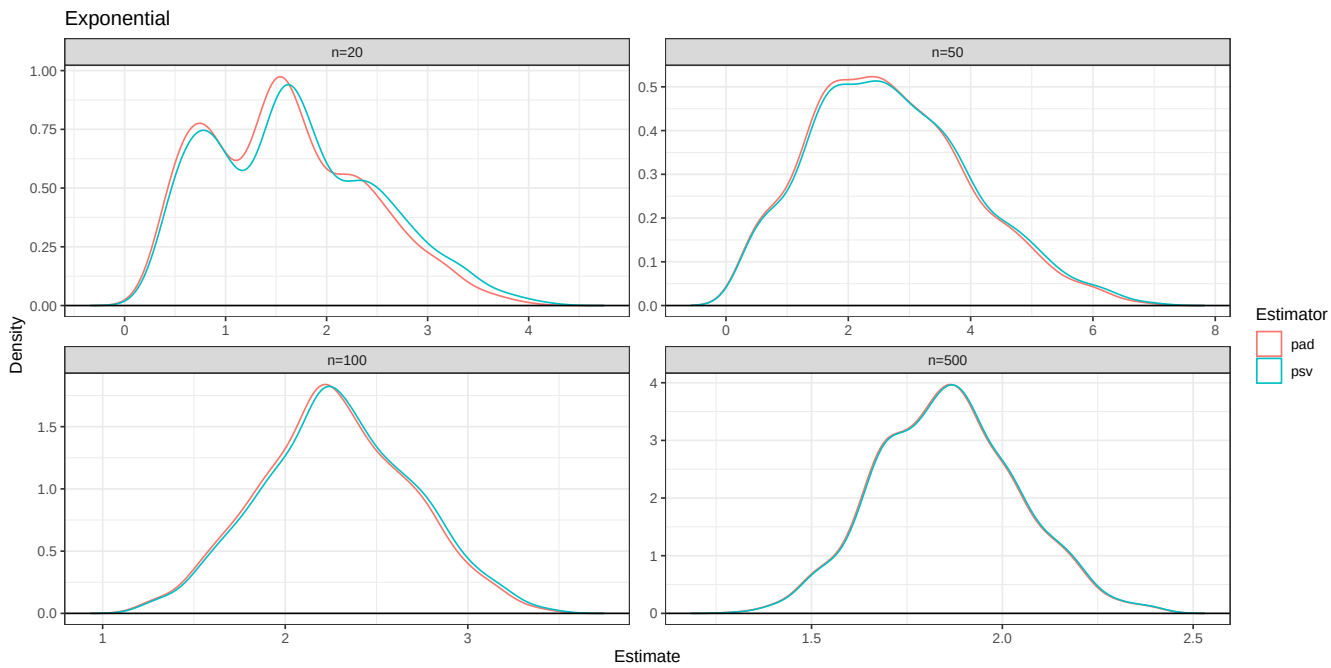

```r
ggdat.nb <- tibble(
Estimator = rep(c("psv", "pad"), each = 1000, times = 4),
n = factor(rep(c("n=20", "n=50", "n=100", "n=500"), each = 2000),
levels = c("n=20", "n=50", "n=100", "n=500")),
Estimate = c(nsv_b[["20"]], nad_b[["20"]],
nsv_b[["50"]], nad_b[["50"]],
nsv_b[["100"]], nad_b[["100"]],
nsv_b[["500"]], nad_b[["500"]])
)

ggplot(ggdat.nb) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Normal") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 23: Normal Density Plot Variance Estimation with Bootstrapping.




```r
set.seed(7272)

ggdat.eb <- tibble(
Estimator = rep(c("psv", "pad"), each = 1000, times = 4),
n = factor(rep(c("n=20", "n=50", "n=100", "n=500"), each = 2000),
levels = c("n=20", "n=50", "n=100", "n=500")),
Estimate = c(esv_b[["20"]], ead_b[["20"]],
esv_b[["50"]], ead_b[["50"]],
esv_b[["100"]], ead_b[["100"]],
esv_b[["500"]], ead_b[["500"]])
)

ggplot(ggdat.eb) +
geom_density_ridges(aes(x = Estimate, y=0, color=`Estimator`), fill=NA) +
geom_hline(yintercept = 0) +
ggtitle("Exponential") +
# lets each facet panel have its own x and y axes instead of forcing them to share the same range
facet_wrap(~ n, scales = "free") +
ylab("Density") +
theme_bw()
```

Figure 24: Exponential Density Plot Variance Estimation with Bootstrapping.



3 Executive Summary

Summarize the work you've done here to provide a brief (200-300 word) summary of the simulation and the results using your work in Lab 11. Your summary should be professional, clear, and all claims should be corroborated with statistical support.

By using histograms as my main visualization of data, I was more able to see the potential skew of sampling distributions and the amount of kurtosus of these distributions, however I did have to rely more on my numerical summaries of tables to gain greater insight on the distributions' accuracy. The current sample variance calculation is a better estimate of variance as opposed to the average squared distance from the mean as sampling distributions of these measures of variance demonstrate that the current sample variance measurement as it is often less bias across the poisson, binomial, normal, and exponential distributions. While the current calculation has worse precision, the accuracy is more helpful in estimating a population's true variance. Additionally, the sampling distributions of the sample variance is much less skewed than the average squared distance from the mean, which is often right-skewed, demonstrating that this parameter estimation often favors a lower population variance than the true parameter. The density plots demonstrate that while tbe average squared distance away from the mean is very precise, this estimate is incredibly likely to report a lower variance than the true parameter, while the sample variance is about equally likely to report lower or higher.

In the bootstrapping portion of the lab, it becomes clear that with one sample, the estimates become much similar as there is a lower amount of variability when all the data comes from one sample taken with replacement. This section also demonstrated the importance of a representative sample. When a sample was less representative, the estimates were about equivalent to each other and were centered more towards the initial sample's mean rather than the population mean. This is especially clear in Figure 13 and **?@fig-fig-binomial-bootstrap**, among many other as this became a common trend. Typically both estimates operated similarly, only really deviating from each other when the number of replacements was lower. When the number of replacements is 500, the density plots of both estimates were basically overlapping. This demonstrates that bootstrapping and more limited data that could possibly be less representative makes bias differences between these estimates less prominent.

4 Citations

I used the tidyverse package (Wickham et al. 2019) to read the csv, and both the tidyverse package (Wickham et al. 2019) and the patchwork package (Pedersen 2025) to create graphs, the knitr package (Xie 2025) to help organize

and reference graphs, the ggridges package (Wilke 2025) to help create the density plots, all while using R (R Core Team 2025) as my coding language.

References

Pedersen, Thomas Lin. 2025. *Patchwork: The Composer of Plots*. https://doi.org/10.32614/CRAN.package.patchwork.

R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. https://www.R-project.org/.

Wickham, Hadley, Mara Averick, Jennifer Bryan, Winston Chang, Lucy D'Agostino McGowan, Romain François, Garrett Grolemund, et al. 2019. "Welcome to the tidyverse." *Journal of Open Source Software* 4 (43): 1686. https://doi.org/10.21105/joss.01686.

Wilke, Claus O. 2025. *Ggridges: Ridgeline Plots in 'Ggplot2'*. https://doi.org/10.32614/CRAN.package.ggridges.

Xie, Yihui. 2025. *Knitr: A General-Purpose Package for Dynamic Report Generation in R*. https://yihui.org/knitr/.